Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Информационная безопасность

Лабораторная работа № 1

Разработка защищённого REST API с интеграцией в CI/CD

Черновик отчёта

Выполнил: Жук Иван Александрович

Группа: P3415

2026

Состояние работы

Репозиторий: https://github.com/XyL1GaN4eG/infosec-1st

Успешный pipeline: https://github.com/XyL1GaN4eG/infosec-1st/actions/runs/42666800103. В запуске пройдены тесты, SpotBugs/FindSecBugs и OWASP Dependency-Check; отчёты SAST и SCA доступны в артефактах запуска.

Локальная проверка

mvn test: 3 интеграционных теста пройдены, ошибок нет. Проверены регистрация и вход, BCrypt, доступ с токеном и без него, неверный, поддельный и просроченный JWT, экранирование ответа и попытка SQL-инъекции.

Проверка с PostgreSQL в отдельном контейнере: регистрация — 201, запрос без токена — 401, запрос с токеном — 200 и пустой список.

mvn compile spotbugs:check: замечаний не обнаружено.

Текст README.md проекта

Лабораторная работа № 1. Защищённый REST API

Учебное приложение на Java 17, Spring Boot, Hibernate и PostgreSQL. Репозиторий (https://github.com/XyL1GaN4eG/infosec-1st).

API

POST /auth/register • Регистрация; возвращает JWT в теле ответа • Открытый

POST /auth/login • Вход; возвращает JWT в теле ответа • Открытый

GET /api/data • Список записей { "id": 1, "data": "..." } • JWT в заголовке Authorization: Bearer <token>

Для регистрации и входа передайте JSON {"login":"testuser","password":"password123"}. Логин и пароль должны состоять из 5–50 символов и не быть пустыми. При повторной регистрации сервер возвращает 409; при неверных учётных данных или токене — 401; при некорректных данных — 400. В новой базе список данных пуст ([]).

Запуск

Требуются JDK 17+, Maven и Docker Compose.

1. Скопируйте .env.example в .env. Если файл уже есть, проверьте, что POSTGRES_HOST=localhost для запуска приложения на хосте. При необходимости поменяйте остальные параметры.

2. Замените JWT_SECRET на случайный ключ в Base64 длиной не менее 32 байт, например: openssl rand -base64 32. Пример в .env.example годится только для локальной разработки.

3. Запустите базу и приложение:

```
docker compose up -d
set -a
source .env
set +a
mvn spring-boot:run
```

PostgreSQL слушает локальный порт из POSTGRES_PORT, внутри контейнера используется 5432. .env не загружается в Spring автоматически: приведённый выше source экспортирует значения. JWT_EXPIRATION задаётся в миллисекундах.

Если база создана предыдущей версией проекта, тип product_table.id мог быть строковым: его необходимо мигрировать в bigint с генерацией идентификаторов либо создать новую базу. Не удаляйте старые данные без резервной копии.

Проверка API

```
# В новой базе регистрируем пользователя и получаем токен.
curl -i -X POST http://localhost:8080/auth/register \
-H 'Content-Type: application/json' \
-d '{"login":"testuser","password":"password123"}'

# Повторный вход: сохраните токен из тела ответа.
TOKEN=$(curl -sS -X POST http://localhost:8080/auth/login \
-H 'Content-Type: application/json' \
-d '{"login":"testuser","password":"password123"}')

curl -i http://localhost:8080/api/data # 401
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/data # 200, []
```

Интеграционные тесты с временной базой H2 запускаются командой mvn test; они проверяют регистрацию, вход, хэш пароля, SQL-инъекцию в логине, экранирование ответа и отказ при отсутствующем, неверном, поддельном и просроченном токене.

Меры защиты

- SQL-инъекции: Spring Data JPA/Hibernate обращается к базе через ORM, данные не подставляются конкатенацией в SQL.

- XSS: ProductService экранирует текст при формировании DTO ответа, не изменяя запись в базе. API возвращает JSON; в HTML, атрибутах, URL или JavaScript клиент обязан применять экранирование, соответствующее контексту, и не вставлять данные через небезопасный innerHTML.

- Аутентификация: BCrypt хранит вместо пароля его хэш; сервер выдаёт JWT с sub, временем выпуска и истечения, подписанный HS256. Фильтр проверяет подпись и срок действия для /api/. Неверный пароль и неизвестный пользователь дают одинаковый ответ. JWT следует передавать только по HTTPS; для реальной эксплуатации также нужны управление отзывом токенов, ограничение попыток входа и безопасное хранение секретов.

Проверки безопасности и CI

При каждом push и pull request workflow .github/workflows/ci.yml (.github/workflows/ci.yml) запускает тесты, SpotBugs с FindSecBugs (SAST) и OWASP Dependency-Check (SCA). Результаты загружаются как артефакты spotbugs-report и dependency-check-report, даже если соответствующая проверка завершилась ошибкой. SpotBugs завершает шаг с ошибкой при обнаружении замечаний; Dependency-Check — при CVSS ≥ 7. Локальный запуск:

```
mvn -B compile spotbugs:check
mvn -B org.owasp:dependency-check-maven:check -Dformat=HTML -DfailBuildOnCVSS=7
```

Dependency-Check загружает базу NVD; первая загрузка может занять продолжительное время. Workflow кэширует базу для последующих запусков. Для ускорения добавьте ключ NVD в GitHub Actions secret NVD_API_KEY. Без ключа проверка также работает с ограничениями NVD API. Просмотрите HTML-отчёт target/dependency-check-report.html и XML-отчёт target/spotbugsXml.xml; обнаруженные уязвимости необходимо устранить или обоснованно обработать, а не скрывать без проверки.

Успешный запуск pipeline: run 42666800103 (https://github.com/XyL1GaN4eG/infosec-1st/actions/runs/42666800103). В нём пройдены тесты, SpotBugs/FindSecBugs и OWASP Dependency-Check; отчёты доступны в артефактах запуска spotbugs-report и dependency-check-report.